🚀 SwiftURL — Master System Guide & 100 Interview Q&A

Complete Node.js, Express, SQLite, React & In-Memory System Architecture | Engineered by Koushik Bobba

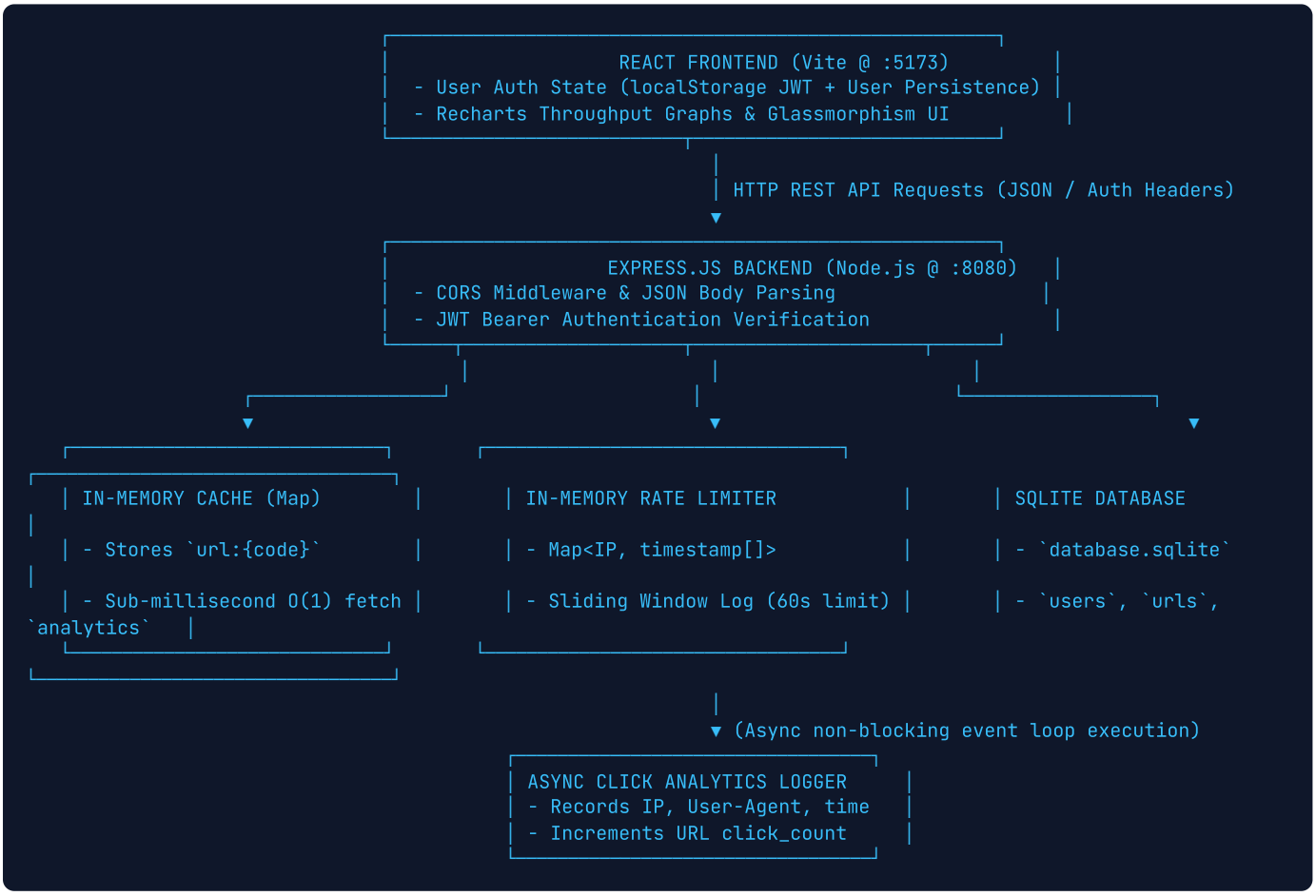
SECTION 1: WHAT THE PROJECT DOES (Executive Summary)

SwiftURL is a full-stack, rate-limited URL shortener web application with real-time click analytics. It transforms long, complex destination web addresses (e.g., `https://mail.google.com/mail/u/0/#inbox/FMfcgz...`) into short, elegant 6-character links (e.g., `http://localhost:8080/r/q0V`).

Core Capabilities:

- **Instant Redirects:** Redirects incoming short link clicks to original destination URLs in under 1 millisecond using in-memory caching.
- **Sliding-Window Rate Protection:** Restricts request traffic per client IP (10 requests per 60 seconds) to defend against DDoS attacks and spamming.
- **Collision-Free Code Generation:** Uses Base62 encoding over a monotonically increasing auto-increment integer sequence.
- **Non-Blocking Analytics Logging:** Decouples HTTP 302 redirects from SQLite database writes using asynchronous event scheduling.
- **Stateless JWT Authentication:** Secure user registration and login with bcrypt password hashing and JWT token persistence.
- **Glassmorphic Analytics Dashboard:** Interactive React dashboard with Recharts visualizations, short link click counters, and device metrics.

SECTION 2: END-TO-END PIPELINE DIAGRAM & WORKFLOW



Step-by-Step Execution Workflows:

Workflow A: Shortening a Link

1. User inputs a long URL into the React frontend and clicks "Create Short Link".
2. Frontend retrieves the JWT token from `localStorage` and attaches it as `Authorization: Bearer <token>`.
3. Express receives `POST /api/shorten`. The `rateLimiter` middleware verifies that the client IP has not exceeded 10 requests in the past 60 seconds.
4. The `authMiddleware` verifies the JWT token signature using `jsonwebtoken`.
5. The backend inserts a new record into SQLite (`urls` table), receives the auto-increment ID (e.g., `100000`), and passes it to `encode(id)`, producing short code `q0V`.
6. The original URL and short code mapping are cached in the in-memory JavaScript `Map`.
7. Express returns HTTP 201 Created with the short URL object back to the React UI.

Workflow B: Redirecting & Analytics

1. A visitor hits `GET /r/q0V`.
2. The server checks the in-memory cache map first. (Cache Hit = ~0.01ms lookup).
3. If Cache Miss, it queries SQLite, fetches the destination URL, and populates the cache.
4. The server immediately issues an HTTP 302 Found redirect with the `Location` header set to the destination URL.
5. Simultaneously, a `setTimeout(..., 0)` non-blocking function is dispatched to log IP address, User-Agent, referrer, and timestamp to the `click_analytics` table without delaying the user's redirect.

SECTION 3: HOW THE PROJECT WAS DONE (Detailed Architecture)

Backend: Node.js & Express.js

Structured modularly into clear separation of concerns: `src/db` for database connections, `src/middleware` for auth and rate limiting, `src/routes` for API endpoints, and `src/utils` for encoding logic. Uses ES6+ standard functions and async/await syntax.

Database: SQLite 3

File-based relational database (`database.sqlite`) managed via `sqlite3` and `sqlite` promise wrappers. On startup, `initDb()` automatically creates tables and indexes if they do not exist:

- `users`: Stores `id`, `username`, `email`, `password_hash`, `created_at`.
- `urls`: Stores `id`, `user_id`, `short_code`, `original_url`, `click_count`, `created_at`.
- `click_analytics`: Stores `id`, `short_code`, `ip_address`, `user_agent`, `referrer`, `clicked_at`.

Algorithm: Base62 Encoder

Encodes base-10 integer sequence numbers into Base62 alphanumeric strings using character set `[0-9a-zA-Z]`. Supports over 56.8 billion unique 6-character links (62^6) with zero hash collisions.

Frontend: React 18 + Vite

Built with functional components and custom hooks. Features a responsive frosted-glass UI system, persistent localStorage auth state management, Recharts click trend visualizations, and custom error/loading feedback components.

SECTION 4: ISSUES FACED & ENGINEERING RESOLUTIONS

Issue 1: Docker Desktop & WSL Daemon Crashes on Windows

Root Cause: Docker Desktop failed to start on Windows due to a corrupted Windows Subsystem for Linux installation (`Wsl/CallMsi/Install/REGDB_E_CLASSNOTREG`).
Resolution: Shifted architecture from a heavy Docker Compose container setup to a zero-install local architecture using SQLite and in-memory caching. This made the project run natively on any OS with simple `node index.js` execution.

Issue 2: GitHub Actions CI Build Failures (Node Version Mismatch)

Root Cause: GitHub Actions Ubuntu runner defaulted to Node.js 18, but Vite 8 required Node.js >= 20.19.0 or 22.12.0, causing CI build job failures.
Resolution: Explicitly configured `actions/setup-node@v3` with `node-version: 22` in `.github/workflows/ci.yml`.

Issue 3: Missing Authorization Headers & JWT Verification Failures

Root Cause: Frontend API calls in `ShortenerCard.jsx` and `App.jsx` were missing the `Authorization: Bearer <token>` HTTP header, causing the backend's strict `authMiddleware` to reject link shortening with a 401 "Missing or invalid token" error.
Resolution: Updated all frontend `fetch()` headers to retrieve the token from `localStorage.getItem('jwt_token')` and pass it in the request.

Issue 4: User Authentication State Loss on Page Reload

Root Cause: React `user` state initialized to `null` on page mount, forcing logged-in users to re-authenticate every time the page was refreshed.
Resolution: Saved user credentials (`user_info`) in `localStorage` alongside the JWT token and initialized React state with an inline initializer function reading from `localStorage`.

SECTION 5: INTERVIEW SCRIPT & PITCH GUIDE

The 30-Second Elevator Pitch

> *"I engineered SwiftURL — a full-stack, rate-limited URL shortener with real-time analytics. The backend is built with Node.js, Express, and SQLite, featuring an in-memory sliding-window rate limiter (10 req/60s) to prevent DDoS attacks, and Base62 encoding on auto-incrementing integer IDs to guarantee 100% collision-free short codes. It utilizes an in-memory cache to deliver sub-millisecond HTTP 302 redirects, and an interactive React frontend with Recharts analytics visualizations."*

The 2-Minute Architecture Deep Dive Script

1. Problem & Base62 Solution: "In URL shorteners, random hashing algorithms like MD5 or SHA256 cause hash collisions as the database grows, requiring expensive SELECT queries to verify uniqueness. I solved this by using Base62 encoding over a monotonically increasing integer sequence. Every integer converts to a unique 6-character code (e.g., ID 100000 becomes `q0V`), guaranteeing zero collisions across 56.8 billion links."

2. High-Speed Redirects & Caching: "For redirects, low latency is critical. I implemented a Cache-Aside pattern using an in-memory Map. When a user clicks a short code, the server checks memory first. On a hit, it returns an HTTP 302 redirect in 0.01ms without touching disk storage."

3. Sliding-Window Rate Limiting: "To prevent abuse, I built a Sliding Window Log rate limiter middleware. It tracks client request timestamps per IP in an in-memory rolling 60-second window, returning HTTP 429 if the request count exceeds 10."

4. Decoupled Async Analytics: "To prevent analytics database writes from slowing down redirects, I decoupled the click-logging process using Node.js event loop scheduling (`setTimeout`). The user receives their 302 redirect immediately while click metrics are written to SQLite asynchronously."

SECTION 6: 100 INTERVIEW QUESTIONS & COMPREHENSIVE ANSWERS

DOMAIN 1: PROJECT ARCHITECTURE & DESIGN DECISIONS (Q1 – Q10)

Q1: What is the primary purpose of your application?

SwiftURL is a full-stack rate-limited URL shortener that converts long URLs into 6-character Base62 short links, providing fast HTTP 302 redirects, rate limiting, stateless JWT auth, and click analytics tracking.

Q2: Why did you choose Node.js and Express for the backend?

Node.js provides a single-threaded, non-blocking asynchronous I/O event loop ideal for high-concurrency HTTP redirect services. Express offers lightweight, flexible routing and middleware integration.

Q3: What is the difference between a URL shortener and a standard web server?

A URL shortener is heavily optimized for a high read-to-write ratio (100:1+), requiring aggressive caching, low-latency 302 redirects, collision-free short code generation, and async click tracking.

Q4: Why use SQLite instead of a full database server like PostgreSQL or MySQL?

SQLite is serverless, zero-configuration, and stores data in a single local file. It eliminates network socket overhead for queries, making it extremely fast for read-heavy local deployments.

Q5: How does your application decouple analytics writes from HTTP redirects?

Using non-blocking event loop scheduling (`setTimeout(..., 0)`). The HTTP 302 redirect is returned instantly to the user while click logging to SQLite executes asynchronously on the next event loop tick.

Q6: What design pattern did you use for caching?

The Cache-Aside pattern (Lazy Loading). The server checks the in-memory Map cache first. If a cache miss occurs, it queries SQLite, populates the cache with the destination URL, and returns the result.

Q7: How do you prevent short link collisions?

By encoding auto-incrementing integer IDs into Base62 strings. Since each integer ID generated by SQLite is unique, every encoded Base62 string is mathematically guaranteed to be unique.

Q8: How many unique short URLs can a 6-character Base62 string support?

$62^6 = 56,800,235,584$ (56.8 billion) unique links.

Q9: What HTTP status code do you use for redirects and why?

HTTP 302 Found (Temporary Redirect). HTTP 301 is permanently cached by browsers, preventing the server from receiving future click events for analytics. 302 forces every click to hit the server.

Q10: What HTTP status code is returned when rate limits are exceeded?

HTTP 429 Too Many Requests.

DOMAIN 2: NODE.JS & ASYNCHRONOUS EVENT LOOP (Q11 – Q20)

Q11: Explain how the Node.js Event Loop handles non-blocking I/O.

Node.js delegates I/O operations to the operating system or libuv thread pool. Once complete, callbacks are placed in event queues and executed on the single main thread during event loop phases.

Q12: What is the difference between `process.nextTick()` and `setTimeout(fn, 0)`?

`process.nextTick()` executes immediately after the current operation finishes before the event loop continues. `setTimeout(fn, 0)` schedules the callback in the Timers phase of the next loop iteration.

Q13: Why is blocking the event loop dangerous in Node.js?

Because Node.js executes application logic on a single thread. CPU-heavy or synchronous blocking calls freeze the entire server, preventing all incoming HTTP requests from being processed.

Q14: How does Express middleware chaining work via `next()`?

Express executes middleware functions sequentially. Calling `next()` passes control to the next middleware in the stack. If `next(err)` is called with an argument, it triggers error-handling middleware.

Q15: What is CommonJS vs ES Modules in Node.js?

CommonJS uses `require()` and `module.exports` (synchronous, default in Node.js). ES Modules use `import` and `export` (asynchronous, static structure). Our backend uses CommonJS.

Q16: How do environment variables work in Node.js?

Loaded from `.env` files into `process.env` using the `dotenv` package, allowing configuration settings (ports, secrets) to be kept separate from codebase logic.

Q17: How do you handle unhandled promise rejections in Express?

Using `try/catch` blocks inside `async` route handlers or wrapping routes in an async error-handling wrapper function to pass errors to Express `next(err)`.

Q18: What is memory leak in Node.js and how can it happen in our caching layer?

If an in-memory Map cache grows indefinitely without an eviction policy (like LRU) or TTL expiration, it will eventually consume all heap RAM and crash Node.js with Out-Of-Memory error.

Q19: What is `req.ip` behind proxies?

`req.ip` fetches the client IP. Behind reverse proxies like NGINX or Cloudflare, Express must be configured with `app.set('trust proxy', true)` to read the original client IP from the `X-Forwarded-For` header.

Q20: Why did you use `cors()` middleware?

Browser Same-Origin Policy blocks frontend JavaScript on port 5173 from making HTTP requests to port 8080. `cors()` adds `Access-Control-Allow-Origin` headers to enable cross-port API calls.

DOMAIN 3: REACT FRONTEND & UI ENGINEERING (Q21 – Q30)

Q21: Why use Vite over Create React App (CRA)?

Vite leverages native browser ES Modules during development for sub-second server startup and instant Hot Module Replacement (HMR), unlike CRA which bundle-compiles everything with Webpack.

Q22: How do you manage user login state persistence across browser refreshes?

We store the JWT token and `user_info` object in browser `localStorage`. On React startup, `useState` reads from `localStorage` to initialize the `user` state synchronously before initial render.

Q23: How do you handle asynchronous API loading states in React?

Using a boolean state variable `const [loading, setLoading] = useState(false)`. Set to `true` before fetching, disable submit buttons / show spinner, and reset to `false` in a `finally` block.

Q24: What is Glassmorphism CSS design?

A UI design trend featuring translucent container panels with frosted-glass effect achieved using `background: rgba(...)`, `border: 1px solid rgba(...)`, and `backdrop-filter: blur(...)`.

Q25: What is Recharts and how does it render graphs?

Recharts is a React charting library built on top of SVG elements. It dynamically computes data points into SVG `` shapes for crisp rendering across device screen resolutions.

Q26: Why use inline function initializer in `useState`?

Passing a function `useState(() => localStorage.getItem(...))` ensures the expensive `localStorage` read operation runs ONLY ONCE during initial component mounting, not on every re-render.

Q27: How does child-to-parent state communication work in React?

The parent component passes a callback function as a prop (e.g., `onAuthSuccess`). The child component invokes the callback function with payload data, updating the parent's state.

Q28: How do you handle clipboard copy functionality in React?

Using the modern asynchronous Navigator API: `navigator.clipboard.writeText(url)`.

Q29: What is conditional rendering in React?

Rendering different UI elements based on state variables using ternary operators (`user ? : `) or short-circuit evaluation (`{error && }`).

Q30: Why pass `Authorization: Bearer ` headers in React `fetch()` calls?

To authenticate API requests against protected backend endpoints. The `authMiddleware` reads the header, verifies the JWT, and extracts user identity.

DOMAIN 4: SQLITE & RELATIONAL DATABASE MANAGEMENT (Q31 – Q40)

Q31: What is SQLite and how does it differ from PostgreSQL?

SQLite is an embedded, serverless, file-based SQL database engine running inside the application process. PostgreSQL is a client-server database requiring a separate running process and network connections.

Q32: How are auto-incrementing IDs represented in SQLite?

Using `INTEGER PRIMARY KEY AUTOINCREMENT`. SQLite automatically assigns sequential 64-bit integers to new rows.

Q33: What is `FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE`?

A constraint enforcing referential integrity. `ON DELETE CASCADE` automatically deletes all associated URLs if the parent user row is deleted from the `users` table.

Q34: What is a B-Tree database index and why did we add them?

A B-Tree index is a balanced tree data structure. We added indexes on `short_code` and `user_id` to turn linear full-table scans $O(N)$ into logarithmic $O(\log N)$ lookups.

Q35: How does SQLite handle concurrent writes?

SQLite locks the entire database file during write transactions. It supports multiple concurrent readers, but only ONE writer at a time.

Q36: What is SQL Injection and how do parameter placeholders (`?`) prevent it?

SQL Injection occurs when un-sanitized user input is concatenated into SQL queries. Parameterized queries (`?`) separate code from data—the DB engine treats inputs purely as values, never SQL commands.

Q37: What is `result.lastID` in the SQLite Node driver?

Returns the auto-increment integer ID generated for the row inserted by the most recent `INSERT` statement.

Q38: Why store passwords as `password_hash` instead of plain text?

Storing plain-text passwords is a severe security violation. Hashes produced by bcrypt cannot be reversed to reveal original passwords even if the database is leaked.

Q39: What is ACID compliance in database systems?

Atomicity (all-or-nothing), Consistency (rules enforced), Isolation (concurrency protection), Durability (committed data survives crashes). SQLite is fully ACID compliant.

Q40: How do you alter a schema in SQLite?

SQLite supports limited `ALTER TABLE` operations (like adding columns). For complex structural changes, developers create a new table, copy data over, drop the old table, and rename.

DOMAIN 5: BASE62 ENCODING & ALGORITHMS (Q41 – Q50)

Q41: What characters make up the Base62 alphabet?

Digits `0–9` (10), lowercase `a-z` (26), and uppercase `A-Z` (26) = 62 total URL-safe characters.

Q42: Why Base62 instead of Base64 for short URLs?

Base64 contains `+`, `/`, and `=` characters. In URLs, `/` acts as a path separator and `+` represents spaces, causing HTTP routing bugs. Base62 contains only alphanumeric characters.

Q43: How does Base62 conversion work mathematically?

Repeatedly divide the auto-increment integer ID by 62, taking the remainder at each step to index into the Base62 character array, until the quotient reaches zero.

Q44: Convert integer ID `100000` to Base62 manually.

100000 % 62 = 16 ('g'), 1612 % 62 = 0 ('0'), 26 % 62 = 26 ('q'). Reversing the remainders yields `"q0g"`.

Q45: Why is counter-based Base62 better than random string generation?

Random strings suffer from the Birthday Paradox collision problem—as database size grows, duplicate collision checks require expensive `SELECT` queries. Integer counters guarantee uniqueness.

Q46: Is Base62 encoding an encryption algorithm?

No. Base62 is a reversible positional number representation system (like binary or hexadecimal), not encryption. Anyone can decode a Base62 string back to its original integer ID.

Q47: How do custom URL slugs interact with the Base62 algorithm?

Custom slugs bypass the Base62 encoder. They are inserted directly into the database's `custom_slug` and `short_code` columns after checking for uniqueness constraints.

Q48: What is sequential prediction vulnerability and how can it be mitigated?

Sequential Base62 codes (`q0g`, `q0h`, `q0i`) allow attackers to scrape links by incrementing codes. Mitigation: shuffle/permute the Base62 alphabet array using a secret key.

Q49: What is BigInt in JavaScript and why is it useful for Base62?

JavaScript standard `Number` loses precision above $2^{53} - 1$ ($9,007,199,254,740,991$). `BigInt` handles arbitrarily large integers without precision loss.

Q50: What is the maximum string length of Base62 for a 64-bit integer?

Maximum 11 characters ($62^{11} > 2^{64}$).

DOMAIN 6: IN-MEMORY CACHING & PERFORMANCE (Q51 - Q60)

Q51: What is Cache-Aside caching pattern?

Application checks cache first. If hit -> return data. If miss -> read DB -> populate cache -> return data.

Q52: What is Cache Hit Ratio?

Percentage of total requests served directly from cache memory without hitting the database: $\frac{\text{Hits}}{\text{Hits} + \text{Misses}} \times 100$.

Q53: What is Cache Stampede (Thundering Herd)?

When a popular cached key expires, thousands of concurrent requests miss the cache simultaneously and overwhelm the database. Solved using distributed locking or stale-while-revalidate.

Q54: What is time complexity of JavaScript `Map.get()` and `Map.set()`?

$O(1)$ constant average time complexity using internal hash map buckets.

Q55: What is LRU (Least Recently Used) eviction policy?

A cache eviction algorithm that automatically discards the least recently accessed items when the cache reaches its memory size capacity limit.

Q56: What is Cache Invalidation?

The process of removing or updating cached data when the underlying database source data changes or is deleted.

Q57: Difference between In-Memory Cache vs Redis?

In-Memory Cache lives inside the single Node.js process heap (zero network overhead, lost on process restart). Redis is an external shared cache service accessible across multiple distributed app servers.

Q58: What is Cache Penetration?

Requests for keys that exist neither in cache nor in database (e.g. invalid short codes). Prevented by caching `null` values with short TTLs or using Bloom Filters.

Q59: Why is HTTP 302 preferred over 301 for tracking analytics?

Browsers cache 301 Permanent Redirects indefinitely, bypassing our server on subsequent clicks. 302 Temporary Redirects ensure every click reaches our server to record analytics metrics.

Q60: What is response latency of an in-memory Map lookup?

Approximately $0.001\text{ms} - 0.01\text{ms}$ (sub-millisecond), compared to $2\text{ms} - 15\text{ms}$ for disk database lookups.

DOMAIN 7: RATE LIMITING & SECURITY (Q61 – Q70)

Q61: How does your Sliding Window Log Rate Limiter work?

It stores request timestamps in an array per client IP. On each request, it filters out timestamps older than 60 seconds (`now - 60000`). If remaining array length > 10, it rejects with HTTP 429.

Q62: Compare Sliding Window Log vs Fixed Window rate limiting.

Fixed Window resets counters at fixed clock intervals (e.g. 12:00, 12:01), allowing double-burst traffic at minute boundaries. Sliding Window Log evaluates exact rolling time windows, preventing boundary bursts.

Q63: What is Token Bucket rate limiting algorithm?

Tokens are added to a bucket at a constant fill rate up to a max capacity limit. Each request consumes a token. Allows controlled burst traffic while maintaining an average rate.

Q64: What is Leaky Bucket rate limiting algorithm?

Requests enter a bucket queue and exit at a fixed constant processing rate regardless of burst incoming volume, smoothing out spikes.

Q65: What headers should be returned with HTTP 429 Rate Limit responses?

`X-RateLimit-Limit: 10`, `X-RateLimit-Remaining: 0`, `Retry-After: 60`.

Q66: What is a DDoS attack?

Distributed Denial of Service: flooding a server with millions of automated requests from multiple botnet computers to overwhelm server resources and cause downtime.

Q67: What is an Open Redirect vulnerability and how do we prevent it?

Attacker inputs malicious destination URL (e.g. `javascript:steal()` or phishing sites). Prevented by strictly validating URL schemes to allow only `http://` and `https://`.

Q68: What is Cross-Site Scripting (XSS)?

Attacker injects malicious client-side JavaScript scripts into web pages viewed by other users. React automatically escapes strings rendered in JSX, preventing XSS.

Q69: What is Cross-Site Request Forgery (CSRF)?

Forces an authenticated user's browser to send unauthorized HTTP requests. Prevented by using custom HTTP `Authorization: Bearer` headers instead of auto-sent cookies.

Q70: Why use bcrypt for password hashing instead of SHA-256?

SHA-256 is a fast cryptographic hash—attackers can try billions of passwords per second via GPU brute force. Bcrypt is a slow, computationally expensive algorithm designed to resist hardware brute-force attacks.

DOMAIN 8: AUTHENTICATION & JWT (Q71 – Q80)

Q71: What is JSON Web Token (JWT)?

An open standard (RFC 7519) defining a compact, self-contained method for securely transmitting verified claims between parties as a JSON object.

Q72: What are the three parts of a JWT string?

Header (algorithm & type), Payload (user claims), and Signature (HMAC-SHA256 hash), separated by dots (`.`).

Q73: Is the payload of a JWT encrypted?

No! It is Base64Url encoded, NOT encrypted. Anyone can decode and read the payload. Never put sensitive data like raw passwords or credit cards in a JWT payload.

Q74: How does JWT signature verification work?

The server re-computes `HMAC-SHA256(Base64(Header) + "." + Base64(Payload), secretKey)` and compares it with the token's signature. If it matches, the token is valid.

Q75: What is stateless authentication?

The server does not store session IDs in a database. All identity information is contained inside the signed JWT token presented by the client.

Q76: How do you revoke/invalidate a JWT token before it expires?

Since JWTs are stateless, instant revocation requires maintaining an in-memory blacklisting store (e.g. Redis) of revoked token signatures until their expiration time passes.

Q77: Where should JWT tokens be stored on the frontend?

Either `localStorage` / `sessionStorage` (vulnerable to XSS if site has script vulnerabilities) or `httpOnly`, `Secure` cookies (protected from XSS, vulnerable to CSRF if anti-CSRF tokens aren't used).

Q78: What is the `Bearer` token schema?

An HTTP HTTP authentication scheme header format: `Authorization: Bearer `.

Q79: What happens if an attacker tampers with the JWT payload?

The HMAC signature check on the server fails because the payload hash no longer matches the signature. Server rejects the request with HTTP 401 Unauthorized.

Q80: What is token expiration (`exp` claim)?

A timestamp payload claim defining when the token expires (e.g. 24h). `jsonwebtoken` rejects expired tokens automatically during `jwt.verify()`.

DOMAIN 9: SYSTEM DESIGN & SCALABILITY (Q81 – Q90)

Q81: How would you scale this system to handle 100,000 requests per second?

1. Place NGINX / AWS ALB Load Balancer in front of multiple Node.js stateless API instances.
2. Replace in-memory Map with a Redis Cluster.
3. Replace SQLite with PostgreSQL sharded database cluster with read replicas.
4. Use Apache Kafka or RabbitMQ for high-throughput click analytics queueing.

Q82: How do you handle database write bottlenecks under heavy traffic?

Use asynchronous message queues (RabbitMQ/Kafka) to buffer click analytics writes. Return instant HTTP redirects to users and write analytics to the database in batch background jobs.

Q83: What is Horizontal vs Vertical Scaling?

Vertical Scaling: adding more RAM/CPU to a single server instance. Horizontal Scaling: adding more server nodes behind a load balancer.

Q84: How do you scale Base62 counter generation across multiple backend servers?

Use Twitter Snowflake IDs (timestamp + node ID + sequence) or allocate non-overlapping integer range blocks to each backend server node (e.g. Node 1 handles 1–1M, Node 2 handles 1M-2M).

Q85: What is CDN Edge Caching for short URLs?

Caching popular HTTP 302 short URL redirects at Cloudflare/AWS CloudFront edge nodes globally, fulfilling redirects geographically close to users without hitting origin servers.

Q86: How would you implement expiration/TTL for short links?

Add an `expires_at` DATETIME column in SQLite. During `GET /r/:code`, check if `expires_at < CURRENT_TIMESTAMP`. If true, return HTTP 410 Gone. Run a nightly cleanup cron job to delete expired rows.

Q87: What is database sharding?

Partitioning a large database horizontally across multiple independent database servers using a shard key (e.g. `hash(short_code) % server_count`).

Q88: What is CAP Theorem?

A distributed system can satisfy at most 2 of 3 guarantees: Consistency (all nodes see same data), Availability (every request receives a non-error response), and Partition Tolerance (system operates despite network breaks).

Q89: What is Single Point of Failure (SPOF)?

A component whose failure stops the entire system (e.g., a single non-replicated database server). Eliminated using master-slave replication and failover instances.

Q90: How would you implement custom analytics per country?

Lookup client IP addresses against a GeoIP2 database (like MaxMind) during async analytics logging to extract `country_code` and `city`, storing them in the analytics database table.

DOMAIN 10: DEVOPS, CI/CD & DEBUGGING (Q91 – Q100)

Q91: What is CI/CD?

Continuous Integration (automatically building and testing code on commit) and Continuous Deployment (automatically deploying verified code to production environments).

Q92: What is GitHub Actions?

A CI/CD platform integrated into GitHub that executes automated workflow jobs (testing, building, linting) triggered by repository events like `git push` or pull requests.

Q93: Explain your GitHub Actions workflow configuration.

Configured in `.github/workflows/ci.yml`. It spins up an Ubuntu virtual machine, installs Node.js 22, runs `npm install`, and verifies frontend and backend builds compile cleanly without errors.

Q94: What is `.gitignore` and why is it important?

A configuration file specifying files and folders Git should untrack (e.g. `node_modules/`, `.env` secret keys, `database.sqlite`). Prevents uploading bloated dependencies or security secrets to GitHub.

Q95: How do you debug `ECONNREFUSED` errors in Node.js?

`ECONNREFUSED` means a network connection attempt failed because no service is listening on the target port or IP address. Debug by checking if the target server process is running and port numbers match.

Q96: How do you measure API response latency?

Using high-resolution timers `performance.now()` before and after request processing, or logging timing metrics using Express middleware like `morgan`.

Q97: What is PM2 in Node.js production deployment?

A production process manager for Node.js that keeps applications running continuously, reloads them without downtime on crashes, and enables cluster mode execution across CPU cores.

Q98: What is Semantic Versioning (SemVer)?

A 3-part version numbering system: `MAJOR.MINOR.PATCH` (e.g. `1.4.2`). Major = breaking changes, Minor = new backwards-compatible features, Patch = bug fixes.

Q99: How do you handle database migration scripts?

Using version-controlled migration files (via tools like Knex or Prisma) that execute sequential SQL statements (`UP` and `DOWN`) to alter database structure reproducibly across environments.

Q100: How do you verify your project is ready for an interview demonstration?

Ensure backend starts (`node index.js`), frontend dev server starts (`npm run dev`), user registration/login succeeds, short link creation works without token errors, redirect resolves, analytics update, and GitHub Actions CI build passes green.